SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	Tharun
Register Number	1924260280
Course Code & Name	
Slot	B
Assignment / Task Title	Advanced Problem Solving and Algorithm Design (Q1–Q7)
GitHub Repository URL	
Submission Date	12 November 2026

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

TASK 1 — Searching a Library Catalogue

Task / Problem Statement

The college library stores the accession numbers of its books in an array (10 books in the trial, 100,000 when fully loaded). Students search for a book by accession number at a kiosk. The program must implement **linear search** on an unsorted array and **binary search** on a sorted array, count the comparisons made by each, and be tested with books at the beginning, middle and end and with a missing book. TODO 1 (the body of `binary_search`) must be completed and executed.

Aim

To implement linear and binary search on a catalogue of accession numbers, count their comparisons, and check experimentally how the cost grows from 10 to 100,000 books.

Concept / Theory

Linear search examines elements one by one, so it needs no ordering: best case 1 comparison, worst case n, average about (n+1)/2 for a present key and n for a missing key, i.e. O(n). **Binary search** works only on a *sorted* array: it compares the key with the middle element and discards the half that cannot contain it, so the range shrinks n, n/2, n/4 ... 1. The worst case is floor($\log_2$ n) + 1 comparisons, i.e. O(log n). Sorting is essential because discarding the left half after a[mid] < key is only valid when every element on the left is smaller. The middle is computed as `low + (high - low) / 2` because `(low + high) / 2` can exceed INT_MAX for very large indices and overflow. The global counter `steps` is increased once for every comparison of a[mid] with the key.

Procedure / Algorithm

1. Set low = 0, high = n - 1 and steps = 0.
2. While low <= high: compute mid = low + (high - low) / 2 and increase steps by one.
3. If a[mid] == key, return mid; if a[mid] < key set low = mid + 1; otherwise set high = mid - 1.
4. When the loop ends with low > high the key is absent, so return -1 (linear search: scan i = 0..n-1, steps++ each time).
5. In main(), search for the first, middle, last and a missing key, print index and comparisons, then run the n = 1,000 / 10,000 / 100,000 experiment.

Program / Source Code

q1.c

```c
#include <stdio.h>
#include <stdlib.h>

int steps; /* comparisons made by the last search */

/* a[] is sorted in ascending order. Returns the index of key, or -1. */
int binary_search(const int a[], int n, int key)
{
    int low = 0, high = n - 1;
    steps = 0;
    while (low <= high) {
        int mid = low + (high - low) / 2;   /* cannot overflow */
        steps++;                            /* one comparison of a[mid] with key */
        if (a[mid] == key)
            return mid;
        if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

/* works on unsorted data; steps counts every comparison */
int linear_search(const int a[], int n, int key)
{
    steps = 0;
    for (int i = 0; i < n; i++) {
        steps++;
```

```
        if (a[i] == key)
            return i;
    }
    return -1;
}

int main(void)
{
    int accession[] = {1021, 1048, 1103, 1190, 1222, 1305, 1377, 1402, 1460, 1533};
    int n = sizeof accession / sizeof accession[0];
    int keys[] = {1021, 1305, 1533, 1400};

    for (int k = 0; k < 4; k++) {
        int pos = binary_search(accession, n, keys[k]);
        printf("Book %d -> index %d, comparisons = %d\n", keys[k], pos, steps);
    }

    printf("\n--- Linear search on the same keys ---\n");
    for (int k = 0; k < 4; k++) {
        int pos = linear_search(accession, n, keys[k]);
        printf("Book %d -> index %d, comparisons = %d\n", keys[k], pos, steps);
    }

    /* Student B experiment: odd numbers 1,3,5,... so even keys are always missing */
    printf("\n--- Experiment (binary search) ---\n");
    printf("%-8s %-12s %-10s %-12s %-10s %-10s\n",
           "n", "succ.avg", "succ.max", "unsucc.avg", "unsucc.max", "log2(n)+1");
    int sizes[] = {1000, 10000, 100000};
    for (int s = 0; s < 3; s++) {
        int N = sizes[s];
        int *a = malloc(N * sizeof(int));
        for (int i = 0; i < N; i++) a[i] = 2 * i + 1;
        long sum = 0; int mx = 0;
        for (int i = 0; i < N; i++) {               /* every present key */
            binary_search(a, N, a[i]);
            sum += steps; if (steps > mx) mx = steps;
        }
        double savg = (double)sum / N;
        long usum = 0; int umx = 0;
        for (int i = 0; i <= N; i++) {              /* N+1 missing keys */
            binary_search(a, N, 2 * i);
            usum += steps; if (steps > umx) umx = steps;
        }
        int pred = 0; for (int t = N; t > 0; t /= 2) pred++;
        printf("%-8d %-12.2f %-10d %-12.2f %-10d %-10d\n",
               N, savg, mx, (double)usum / (N + 1), umx, pred);
        free(a);
    }

    printf("\n--- Experiment (linear search, 1000 sampled keys) ---\n");
    printf("%-8s %-12s %-12s\n", "n", "succ.avg", "unsucc");
    for (int s = 0; s < 3; s++) {
        int N = sizes[s];
        int *a = malloc(N * sizeof(int));
        for (int i = 0; i < N; i++) a[i] = 2 * i + 1;
        long sum = 0;
        for (int t = 0; t < 1000; t++) {
            linear_search(a, N, a[(long)t * (N - 1) / 999]);
            sum += steps;
        }
        linear_search(a, N, 2);
        printf("%-8d %-12.1f %-12d\n", N, sum / 1000.0, steps);
        free(a);
    }
    return 0;
}
```

Output Screenshot

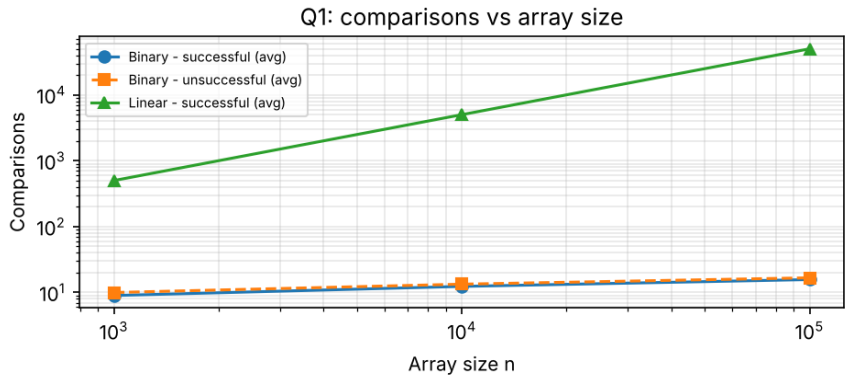
Terminal - gcc output

```
$ gcc q1.c -o q1 && ./q1
Book 1021 -> index 0, comparisons = 3
Book 1305 -> index 5, comparisons = 3
Book 1533 -> index 9, comparisons = 4
Book 1400 -> index -1, comparisons = 4

--- Linear search on the same keys ---
Book 1021 -> index 0, comparisons = 1
Book 1305 -> index 5, comparisons = 6
Book 1533 -> index 9, comparisons = 10
Book 1400 -> index -1, comparisons = 10

--- Experiment (binary search) ---
n        succ.avg     succ.max   unsucc.avg   unsucc.max log2(n)+1
1000     8.99         10         9.98         10         10
10000    12.36        14         13.36        14         14
100000   15.69        17         16.69        17         17

--- Experiment (linear search, 1000 sampled keys) ---
n        succ.avg     unsucc
1000     500.5        1000
10000    5000.0       10000
100000   50000.0      100000
```



Analysis / Discussion

Output explained. The four lines match the expected output: 1021 needs 3 comparisons, 1305 needs 3, 1533 needs 4 and the missing 1400 needs 4. Linear search on the same keys needs 1, 6, 10 and 10, so binary search wins as soon as the key is not at the very start.

Why a missing book costs as much as the worst successful search. A missing key is never found early: the search can only stop when the range becomes empty, which needs the full number of halvings, floor($\log_2$ n) + 1 = 4 for n = 10. A present key can stop earlier, but the deepest present key needs the same 4 steps.

Student A - pseudocode. *Linear:* for i = 0 to n-1: steps++; if a[i] = key return i; end; return -1. *Binary:* low = 0; high = n-1; while low <= high: mid = low + (high-low)/2; steps++; if a[mid] = key return mid; else if a[mid] < key low = mid+1 else high = mid-1; end; return -1.

Student A - best / worst / average.

Search	Best case	Worst case	Average (present key)
Linear	1	n	(n + 1) / 2
Binary	1	floor(log2 n) + 1	about log2 n - 1
Missing key	linear: n	binary: floor(log2 n) + 1	-

Student A - prediction for 100,000 books. $2^{16} = 65{,}536 < 100{,}000 < 2^{17} = 131{,}072$, so binary search needs at most 17 comparisons (about 15.7 on average), while linear search needs 100,000 in the worst case and about 50,000 on average.

Student B - experiment. Sorted arrays of 1,000, 10,000 and 100,000 odd numbers were searched for every present key and for every missing (even) key. Measured worst cases were 10, 14 and 17 comparisons and averages 8.99, 12.36 and 15.69 (successful), 9.98, 13.36 and 16.69 (unsuccessful). The graph (log-log) shows binary search almost flat against the straight line of linear search. The results agree with Student A's predictions (maximum = floor($\log_2$ n) + 1; average about $\log_2$ n - 1).

Joint design question - keep the array sorted if books are added daily? It depends on how many searches happen per day compared with how many books are added. Inserting into a sorted array shifts on average n/2 elements (up to 100,000 moves), whereas appending to an unsorted array is O(1); but every search saves about n/2 - $\log_2$ n comparisons. We would need: number of searches per day, number of new books per day, the catalogue size, and whether insertions can be batched. For example, with 1,000 searches and 50 new books a day, sorting costs about 50 x 50,000 = 2.5 million moves but saves about 1,000 x 50,000 = 50 million comparisons, so keeping it sorted is worthwhile; a good compromise is an unsorted 'new books' buffer searched linearly and merged into the sorted array once a night.

Alternative concept - hash table or interpolation search.

Approach	Time per search	Extra space	Remarks
Binary search (sorted array)	O(log n)	O(1)	needs sorted data
Hash table, key mod size, linear probing	O(1) average, O(n) worst	O(m), m larger than n (e.g. 1.3n to 2n slots)	no sorting; clustering when full
Interpolation search (sorted array)	O(log log n) average, O(n) worst	O(1)	fast for evenly spread keys

Result

Both searches were implemented and TODO 1 was completed; the program printed exactly the expected comparisons (3, 3, 4, 4). The experiment confirmed that binary search needs at most floor($\log_2$ n) + 1 comparisons (17 for 100,000 books) while linear search grows in proportion to n. I learned that a missing key costs the full depth of the search and that the sorted order has a maintenance cost.

TASK 2 — Sorting the Examination Hall Seating Register

Task / Problem Statement

The examination cell needs a seating register sorted by register number. The data may arrive already sorted, nearly sorted (a few late registrations) or in random order. Implement bubble sort (with an early-exit flag), selection sort and insertion sort, count comparisons and swaps (or shifts) for each, and test on sorted, nearly sorted, reverse-sorted and random arrays.

Aim

To implement three elementary sorting algorithms, measure their comparisons and swaps on four kinds of input, and decide which one suits which situation.

Concept / Theory

Bubble sort repeatedly swaps adjacent out-of-order pairs; the *early-exit flag* stops the algorithm when a full pass makes no swap. **Selection sort** finds the smallest remaining element and swaps it into place: always n(n-1)/2 comparisons but at most n-1 swaps. **Insertion sort** takes the next element and shifts larger elements right to open a gap: comparisons about n + (number of inversions), shifts = number of inversions. All three are $O(n^2)$ in the worst case. A sort is **stable** if equal keys keep their original order; bubble and insertion sort are stable, selection sort is not (its long-distance swap can jump an element over an equal one).

Procedure / Algorithm

1. Generate arrays of n = 100, 1,000 and 10,000 register numbers in four orders: sorted, nearly sorted (sorted plus late registrations), reverse and random.
2. Bubble sort: in each pass compare neighbours, swap if a[j] > a[j+1] and set swapped = 1; stop when a pass has swapped = 0.
3. Selection sort: for i = 0..n-2 find the index of the minimum of a[i..n-1] and swap it with a[i].
4. Insertion sort: for i = 1..n-1 keep key = a[i], shift every larger element of a[0..i-1] one place right and drop key in the gap.
5. Reset the counters before each sort, copy the same input for every algorithm and print comparisons and swaps (shifts) in a table.

Program / Source Code

q2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

long cmp, mv;                       /* comparisons, swaps (or shifts) */

void bubble(int a[], int n)         /* with early-exit flag */
{
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++) {
            cmp++;
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                mv++; swapped = 1;
            }
        }
        if (!swapped) break;        /* already sorted: stop early */
    }
}

void selection(int a[], int n)
{
    for (int i = 0; i < n - 1; i++) {
        int m = i;
        for (int j = i + 1; j < n; j++) {
            cmp++;
            if (a[j] < a[m]) m = j;
        }
```

```
        if (m != i) { int t = a[i]; a[i] = a[m]; a[m] = t; mv++; }
    }
}

void insertion(int a[], int n)
{
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0) {
            cmp++;
            if (a[j] > key) { a[j + 1] = a[j]; mv++; j--; }
            else break;
        }
        a[j + 1] = key;
    }
}

void show(const char *tag, const int a[], int n)
{
    printf("%-14s", tag);
    for (int i = 0; i < n; i++) printf("%5d", a[i]);
    printf("\n");
}

void gen(int a[], int n, int order)  /* 0 sorted,1 nearly,2 reverse,3 random */
{
    for (int i = 0; i < n; i++) a[i] = 1000 + i;
    if (order == 1) {                          /* late registrations at the end */
        int k = (n <= 100) ? 5 : 20;
        for (int i = n - k; i < n; i++) a[i] = 1000 + rand() % n;
    } else if (order == 2) {
        for (int i = 0; i < n; i++) a[i] = 1000 + n - 1 - i;
    } else if (order == 3) {
        for (int i = n - 1; i > 0; i--) {
            int j = rand() % (i + 1); int t = a[i]; a[i] = a[j]; a[j] = t;
        }
    }
}

typedef struct { char name; int marks; } Rec;

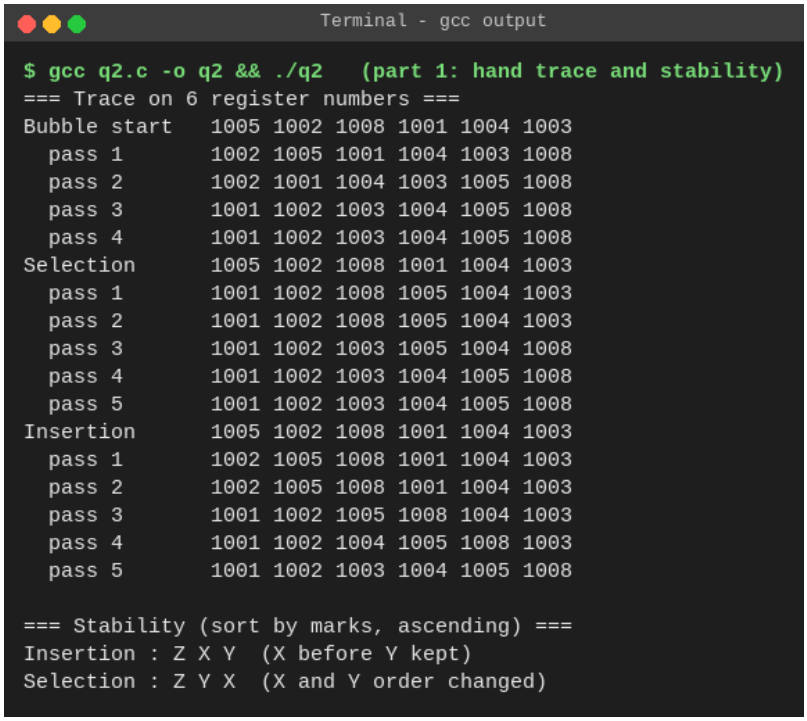
int main(void)
{
    /* ---- hand-trace data (Student A) ---- */
    int t0[6] = {1005, 1002, 1008, 1001, 1004, 1003};
    int t[6];
    printf("=== Trace on 6 register numbers ===\n");
    memcpy(t, t0, sizeof t); show("Bubble start", t, 6);
    for (int i = 0; i < 5; i++) {
        int sw = 0;
        for (int j = 0; j < 5 - i; j++)
            if (t[j] > t[j + 1]) { int x = t[j]; t[j] = t[j + 1]; t[j + 1] = x; sw = 1; }
        char tag[20]; sprintf(tag, "  pass %d", i + 1); show(tag, t, 6);
        if (!sw) break;
    }
    memcpy(t, t0, sizeof t); show("Selection", t, 6);
    for (int i = 0; i < 5; i++) {
        int m = i;
        for (int j = i + 1; j < 6; j++) if (t[j] < t[m]) m = j;
        int x = t[i]; t[i] = t[m]; t[m] = x;
        char tag[20]; sprintf(tag, "  pass %d", i + 1); show(tag, t, 6);
    }
    memcpy(t, t0, sizeof t); show("Insertion", t, 6);
    for (int i = 1; i < 6; i++) {
        int key = t[i], j = i - 1;
        while (j >= 0 && t[j] > key) { t[j + 1] = t[j]; j--; }
        t[j + 1] = key;
        char tag[20]; sprintf(tag, "  pass %d", i); show(tag, t, 6);
    }

    /* ---- stability demo ---- */
    printf("\n=== Stability (sort by marks, ascending) ===\n");
    Rec r[3] = {{'X', 80}, {'Y', 80}, {'Z', 70}};
    Rec s1[3]; memcpy(s1, r, sizeof r);
    for (int i = 1; i < 3; i++) {                      /* insertion */
        Rec k = s1[i]; int j = i - 1;
        while (j >= 0 && s1[j].marks > k.marks) { s1[j + 1] = s1[j]; j--; }
        s1[j + 1] = k;
    }
    printf("Insertion : %c %c %c  (X before Y kept)\n", s1[0].name, s1[1].name, s1[2].name);
    Rec s2[3]; memcpy(s2, r, sizeof r);
    for (int i = 0; i < 2; i++) {                      /* selection */
        int m = i;
        for (int j = i + 1; j < 3; j++) if (s2[j].marks < s2[m].marks) m = j;
        Rec x = s2[i]; s2[i] = s2[m]; s2[m] = x;
    }
```

```
    printf("Selection : %c %c %c  (X and Y order changed)\n", s2[0].name, s2[1].name, s2[2].name);

    /* ---- measurements (Student B) ---- */
    const char *oname[4] = {"Sorted", "Nearly", "Reverse", "Random"};
    int sizes[3] = {100, 1000, 10000};
    srand(7);
    for (int s = 0; s < 3; s++) {
        int n = sizes[s];
        int *a = malloc(n * sizeof(int)), *b = malloc(n * sizeof(int));
        printf("\n=== n = %d  (comparisons / swaps-or-shifts) ===\n", n);
        printf("%-8s | %-17s | %-17s | %-17s\n", "Order", "Bubble", "Selection", "Insertion");
        for (int o = 0; o < 4; o++) {
            gen(a, n, o);
            long c[3], m[3];
            memcpy(b, a, n * sizeof(int)); cmp = mv = 0; bubble(b, n);    c[0] = cmp; m[0] = mv;
            memcpy(b, a, n * sizeof(int)); cmp = mv = 0; selection(b, n); c[1] = cmp; m[1] = mv;
            memcpy(b, a, n * sizeof(int)); cmp = mv = 0; insertion(b, n); c[2] = cmp; m[2] = mv;
            printf("%-8s | %8ld %8ld | %8ld %8ld | %8ld %8ld\n", oname[o],
                   c[0], m[0], c[1], m[1], c[2], m[2]);
        }
        free(a); free(b);
    }
    return 0;
}
```

Output Screenshot

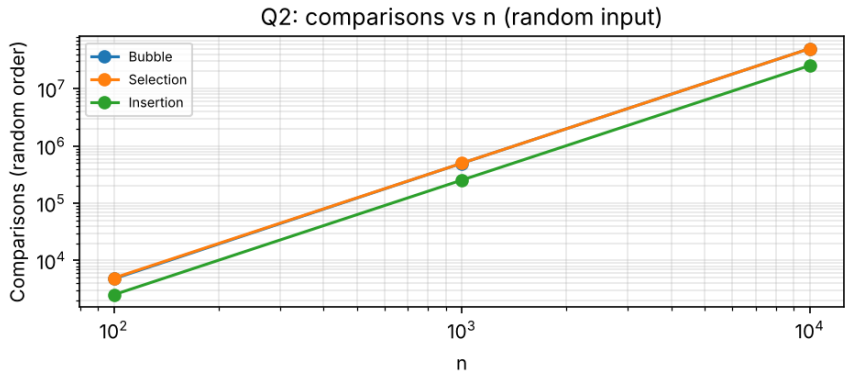


Terminal - gcc output

```
$ ./q2   (part 2: measurements)
=== n = 100  (comparisons / swaps-or-shifts) ===
Order    | Bubble            | Selection         | Insertion
Sorted   |       99        0 |     4950        0 |       99        0
Nearly   |     4625      116 |     4950       72 |      215      116
Reverse  |     4950     4950 |     4950       50 |     4950     4950
Random   |     4797     2433 |     4950       95 |     2529     2433

=== n = 1000  (comparisons / swaps-or-shifts) ===
Order    | Bubble            | Selection         | Insertion
Sorted   |      999        0 |   499500        0 |      999        0
Nearly   |   499290     9888 |   499500      987 |    10887     9888
Reverse  |   499500   499500 |   499500      500 |   499500   499500
Random   |   498720   252394 |   499500      994 |   253389   252394

=== n = 10000  (comparisons / swaps-or-shifts) ===
Order    | Bubble            | Selection         | Insertion
Sorted   |     9999        0 | 49995000        0 |     9999        0
Nearly   | 49990440   121342 | 49995000     9911 |   131341   121342
Reverse  | 49995000 49995000 | 49995000     5000 | 49995000 49995000
Random   | 49982754 25281413 | 49995000     9989 | 25291404 25281413
```



Analysis / Discussion

Student A - hand trace of 6 register numbers (1005 1002 1008 1001 1004 1003). Bubble sort: after pass 1 = 1002 1005 1001 1004 1003 1008, pass 2 = 1002 1001 1004 1003 1005 1008, pass 3 = sorted, pass 4 makes no swap and the flag stops the sort. Selection sort needs 5 passes (1001 1002 1008 1005 1004 1003, then 1003 is brought forward, and so on). Insertion sort grows the sorted left part one element per pass. The program's own trace (first screenshot) confirms the hand trace.

Purpose of the early-exit flag. If a complete pass finds no pair out of order the array is already sorted, so the remaining passes are useless. With the flag, a sorted array costs only n-1 comparisons (measured 99, 999 and 9,999) instead of n(n-1)/2.

Why insertion sort is fast on nearly sorted data. Each element is shifted only past the elements larger than itself, so the work is n plus the number of inversions. With only a few late registrations the inversions are few: for n = 10,000 insertion sort used 131,341 comparisons against 49,995,000 for selection sort.

Stability. Bubble and insertion sort are stable; selection sort is not (demo: records X80, Y80, Z70 became Z Y X). Stability matters for students with equal marks: if the list is already in register-number order and is then sorted by marks, a stable sort keeps equal-mark students in register-number order.

Student B - best algorithm for each input order (n = 10,000). *Sorted:* bubble (with flag) and insertion, 9,999 comparisons and 0 moves; selection still needs 49,995,000. *Nearly sorted:* insertion (131,341 comparisons). *Reverse:* all three need 49,995,000 comparisons, but selection makes only 5,000 swaps against 49,995,000 for the

others, so it is best when writes are expensive. *Random:* insertion needs about half the comparisons (25.3 million versus about 50 million) and its shifts equal the inversions; selection makes the fewest moves (9,989).

Result that differs from expectation. Bubble sort on the nearly sorted data took 49,990,440 comparisons, almost as many as on random data, although the early-exit flag was present. The late registrations are at the end and have small values; bubble sort moves a small value left only one place per pass, so about n passes are needed and the flag never triggers. Insertion sort moves such an element directly to its place.

Joint conclusion. For a register that is sorted except for 20 late registrations I recommend **insertion sort**: nearly linear time (n plus about 20 x n shifts in the worst case), stable, in place and simple. Bubble sort suffers from the slow-moving small values and selection sort ignores the existing order.

Alternative concept - counting sort. If register numbers lie in a known range of size k, subtract the minimum, count how many times each value occurs in an array count[k], turn the counts into positions (prefix sums) and write the values out: time O(n + k) against O(n^2) for the three sorts (for n = 10,000 and k = 1,000 about 11,000 steps instead of 50 million), extra space O(k) (plus an O(n) output array if stability is needed). It is not suitable when k is huge compared with n (for example full 10-digit numbers), when keys are not integers (names, decimals), or when memory is tight.

Result

All three sorts were implemented and measured on four input orders and three sizes. Early-exit bubble sort and insertion sort are linear on sorted data, insertion sort is the best choice for nearly sorted data, and selection sort is attractive only when swaps are costly. I learned that the order of the input can matter more than the algorithm name, and that bubble sort does not benefit from a few small values at the end.

TASK 3 — Merging Two Merit Lists

Task / Problem Statement

Two campuses each produce a merit list sorted in descending order of entrance score. The admissions office must produce one combined list, also in descending order, without sorting again. Implement the merge of two sorted arrays into a third array, handle lists of different lengths and equal scores, verify the result is in order, and complete TODO 1 of `merge()`.

Aim

To merge two descending merit lists in one pass and to analyse how many comparisons and how much memory the merge needs.

Concept / Theory

Merging exploits the fact that both inputs are already sorted: the next output element must be the larger of the two *front* elements, so two indices i and j walk forward through the lists and one comparison decides each output element. When one list is exhausted the other is already in order and is copied without further comparison. Time is O(m + n) and the extra memory is the output array of m + n elements. Using `a[i] >= b[j]` makes the merge **stable** for equal scores (campus A first).

Procedure / Algorithm

1. Set i = 0, j = 0, k = 0 (positions in a[], b[] and out[]).
2. While i < na and j < nb: if a[i] >= b[j] copy a[i++] to out[k++], otherwise copy b[j++] to out[k++].
3. When the loop stops, copy whatever remains of a[] (while i < na).
4. Then copy whatever remains of b[] (while j < nb); only one of the two copy loops can run.
5. Return k = na + nb and verify with a helper that out[] is in descending order.

Program / Source Code

q3.c

```
#include <stdio.h>
#include <stdlib.h>

long cmps;                                  /* comparisons made by merge */

/* a[] and b[] are merit lists sorted in DESCENDING order of score.
   Copies both into out[] in descending order and returns the total count. */
int merge(const int a[], int na, const int b[], int nb, int out[])
{
    int i = 0, j = 0, k = 0;
    cmps = 0;
    while (i < na && j < nb) {              /* both lists still have elements */
        cmps++;
        if (a[i] >= b[j])                   /* >= : on a tie campus A goes first */
            out[k++] = a[i++];
        else
            out[k++] = b[j++];
    }
    while (i < na) out[k++] = a[i++];       /* leftovers of a[] */
    while (j < nb) out[k++] = b[j++];       /* leftovers of b[] */
    return k;
}

static int in_order(const int x[], int n)
{
    for (int i = 1; i < n; i++) if (x[i - 1] < x[i]) return 0;
    return 1;
}

static long qcmps;
static int desc(const void *p, const void *q)
{
    qcmps++;
    return *(const int *)q - *(const int *)p;
}

static void show(const int x[], int n)
```

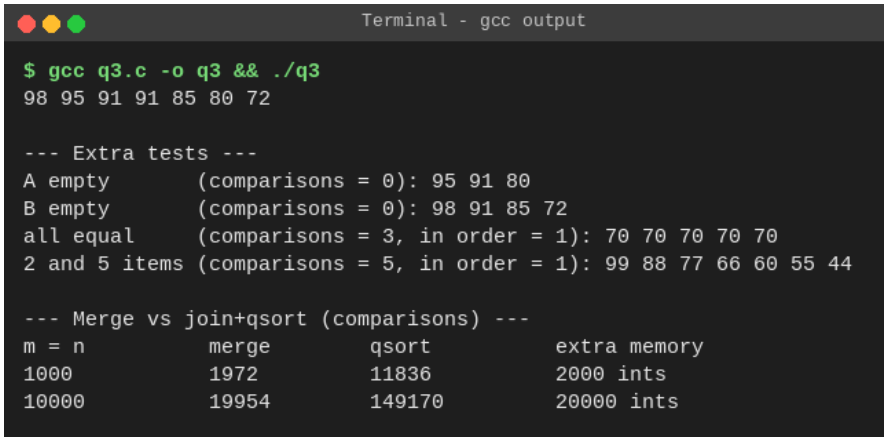
```
{
    for (int i = 0; i < n; i++) printf("%d ", x[i]);
    printf("\n");
}

int main(void)
{
    int campusA[] = {98, 91, 85, 72};
    int campusB[] = {95, 91, 80};
    int merged[7];
    int n = merge(campusA, 4, campusB, 3, merged);
    for (int i = 0; i < n; i++) printf("%d ", merged[i]);
    printf("\n");

    printf("\n--- Extra tests ---\n");
    int empty[1], out[8];
    n = merge(empty, 0, campusB, 3, out);
    printf("A empty       (comparisons = %ld): ", cmps); show(out, n);
    n = merge(campusA, 4, empty, 0, out);
    printf("B empty       (comparisons = %ld): ", cmps); show(out, n);
    int e1[] = {70, 70, 70}, e2[] = {70, 70};
    n = merge(e1, 3, e2, 2, out);
    printf("all equal     (comparisons = %ld, in order = %d): ", cmps, in_order(out, n)); show(out, n);
    int u1[] = {99, 60}, u2[] = {88, 77, 66, 55, 44};
    n = merge(u1, 2, u2, 5, out);
    printf("2 and 5 items (comparisons = %ld, in order = %d): ", cmps, in_order(out, n)); show(out, n);

    printf("\n--- Merge vs join+qsort (comparisons) ---\n");
    printf("%-14s %-12s %-14s %-12s\n", "m = n", "merge", "qsort", "extra memory");
    int sizes[2] = {1000, 10000};
    srand(3);
    for (int s = 0; s < 2; s++) {
        int m = sizes[s];
        int *A = malloc(m * sizeof(int)), *B = malloc(m * sizeof(int));
        int *O = malloc(2 * m * sizeof(int)), *J = malloc(2 * m * sizeof(int));
        A[0] = 100000; B[0] = 99999;
        for (int i = 1; i < m; i++) { A[i] = A[i - 1] - 1 - rand() % 5; B[i] = B[i - 1] - 1 - rand() % 5; }
        merge(A, m, B, m, O);
        long mc = cmps;
        for (int i = 0; i < m; i++) { J[i] = A[i]; J[m + i] = B[i]; }
        qcmps = 0; qsort(J, 2 * m, sizeof(int), desc);
        printf("%-14d %-12ld %-14ld %d ints\n", m, mc, qcmps, 2 * m);
        free(A); free(B); free(O); free(J);
    }
    return 0;
}
```

Output Screenshot



Analysis / Discussion

Output explained. The first line, 98 95 91 91 85 80 72, is exactly the expected output. The extra tests pass: an empty list needs 0 comparisons because the first while loop never starts and the copy loop for the other list does all the work; with all scores equal the merge stays in order; lists of 2 and 5 items also merge correctly.

Student A - number of comparisons. Every comparison writes exactly one element to the output and the comparing loop ends as soon as one list is empty, so the number of comparisons is at most m + n - 1 (worst case: the two lists interleave) and at least min(m, n) (best case: one list is entirely larger than the other).

Student A - merge versus join and sort again.

m = n	Merge (comparisons)	Join + qsort (comparisons)	Observation
1,000	1,972	11,836	sorting needs about 6 times more
10,000	19,954	149,170	sorting needs about 7.5 times more

Student A - extra memory and tests. The merge needs one output array of m + n integers (2,000 or 20,000 ints above); the inputs are not changed. Both special tests (one list empty, all scores equal) are in the output.

Student B - pseudocode. i = j = k = 0; while i < na and j < nb: if a[i] >= b[j] then out[k++] = a[i++] else out[k++] = b[j++]; while i < na: out[k++] = a[i++]; while j < nb: out[k++] = b[j++]; return k.

Student B - each element is handled once. Each element is written to the output after winning exactly one comparison and is never looked at again, so the total number of comparisons can never exceed the number of output elements, m + n - 1. (The element that loses stays in place and meets the next front element of the other list, but it is only written once.) Sorting again would re-examine elements many times.

Student B - copying the remainder. After one list is used up, every remaining element of the other list is already in order and is not larger than anything written so far, so it is copied directly with a simple loop and no comparison.

Student B - equal scores. Equal scores keep their original campus order: because of >= the element of campus A is taken first, then the equal element of campus B. This makes the result predictable and stable, and is fair because the same rule is applied for every tie.

Joint engineering decision - merge in place? The office would prefer an in-place merge when memory is very limited (embedded or huge lists where a third array cannot be allocated) or when one array already has free space at its end. The cost is time and complexity: a simple in-place merge shifts elements and is O(m x n) in the worst case, the code is harder, and stability is harder to guarantee.

Alternative concept - copy and qsort(), or simple in-place merge.

Approach	Time	Extra space	Comment
Merge into a third array	O(m + n)	O(m + n)	fastest, simple
Copy into one array and qsort()	O((m + n) log(m + n))	O(1) to O(m + n) (library dependent)	ignores existing order
Simple in-place merge (shifting)	O(m x n) worst case	O(1)	slow, harder code

Result

The merge was completed with TODO 1 and produced 98 95 91 91 85 80 72 as expected; empty-list, equal-score and unequal-length tests also passed. I learned that using the already sorted order gives O(m + n) work (about 6 to 7.5 times fewer comparisons than qsort here) at the price of m + n extra memory.

TASK 4 — Two-Dimensional Arrays: District Rainfall Records

Task / Problem Statement

Monthly rainfall (mm) for 38 districts over 12 months is stored in rain[38][12]. The disaster management cell needs the annual total per district, the monthly average for the state, the wettest district-month, and the districts whose rainfall fell below 60% of the state average. Declare, initialise and read the array, compute these values and list the deficient districts.

Aim

To store and process tabular data in a two-dimensional array, to understand its row-major layout, and to measure how traversal order affects speed.

Concept / Theory

A 2-D array is an array of rows stored one after another in memory (**row-major order**): rain[0][0..11], then rain[1][0..11], and so on. The address of an element is base + (i x 12 + j) x sizeof(element). Row totals sum along j, column (monthly) averages sum along i. Walking row by row touches neighbouring addresses and uses the CPU cache well (spatial locality); walking column by column jumps a whole row each time. 'Deficient' is interpreted here as: annual total of a district < 60% of the state average annual total per district.

Procedure / Algorithm

1. Declare double rain[38][12] and fill it (rows = districts, columns = months); the program uses generated sample data with two deliberately dry districts.
2. row_totals(): for every row i add the 12 months into tot[i].
3. col_averages(): for every month j add the 38 districts and divide by 38.
4. max_element(): scan all cells, remember the largest value and its row and column.
5. deficient(): average of the totals, limit = 0.6 x average, list every district with tot[i] < limit; then verify on a 3 x 4 array and time row-wise versus column-wise traversal of a 2000 x 2000 array.

Program / Source Code

q4.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define D 38
#define M 12

void read_rain(double r[D][M])                /* initialise with sample data */
{
    for (int i = 0; i < D; i++)
        for (int j = 0; j < M; j++)
            r[i][j] = 20 + (i * 37 + j * 53) % 180 + (i == 5 || i == 21 ? -15 : 0);
    for (int j = 0; j < M; j++) { r[5][j] = 8 + j; r[21][j] = 6 + j; }  /* two dry districts */
}

void row_totals(int rows, int cols, double a[rows][cols], double tot[])
{
    for (int i = 0; i < rows; i++) {
        tot[i] = 0;
        for (int j = 0; j < cols; j++) tot[i] += a[i][j];
    }
}

void col_averages(int rows, int cols, double a[rows][cols], double avg[])
{
    for (int j = 0; j < cols; j++) {
        double s = 0;
        for (int i = 0; i < rows; i++) s += a[i][j];
        avg[j] = s / rows;
    }
}

double max_element(int rows, int cols, double a[rows][cols], int *bi, int *bj)
{
```

```
    double mx = a[0][0]; *bi = *bj = 0;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            if (a[i][j] > mx) { mx = a[i][j]; *bi = i; *bj = j; }
    return mx;
}

int deficient(int rows, double tot[], int list[])  /* annual total < 60% of state average */
{
    double s = 0; int c = 0;
    for (int i = 0; i < rows; i++) s += tot[i];
    double limit = 0.6 * (s / rows);
    for (int i = 0; i < rows; i++) if (tot[i] < limit) list[c++] = i;
    return c;
}

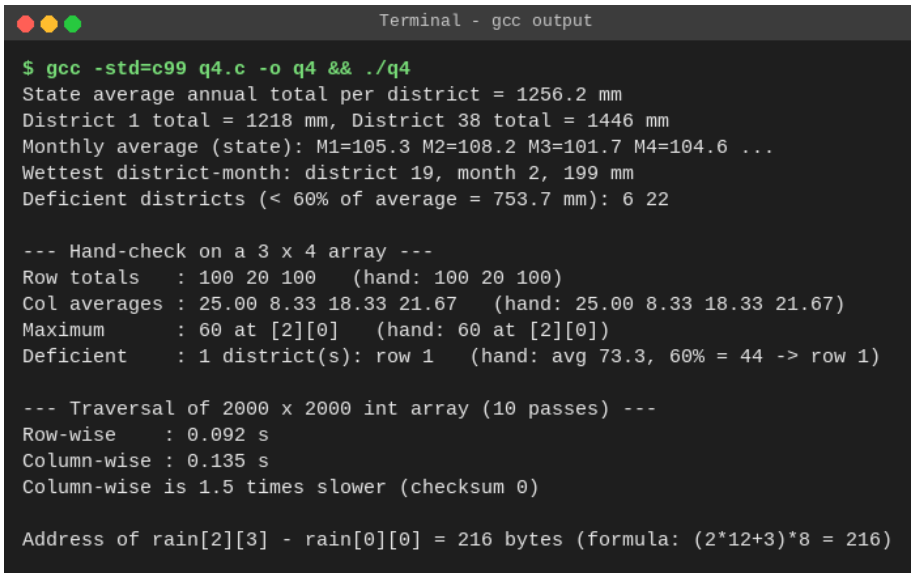
static int big[2000][2000];

int main(void)
{
    static double rain[D][M];
    double tot[D], avg[M]; int list[D], bi, bj;
    read_rain(rain);
    row_totals(D, M, rain, tot);
    col_averages(D, M, rain, avg);
    double mx = max_element(D, M, rain, &bi, &bj);
    int c = deficient(D, tot, list);
    double st = 0; for (int i = 0; i < D; i++) st += tot[i];
    printf("State average annual total per district = %.1f mm\n", st / D);
    printf("District 1 total = %.0f mm, District 38 total = %.0f mm\n", tot[0], tot[37]);
    printf("Monthly average (state): ");
    for (int j = 0; j < 4; j++) printf("M%d=%.1f ", j + 1, avg[j]);
    printf("...\n");
    printf("Wettest district-month: district %d, month %d, %.0f mm\n", bi + 1, bj + 1, mx);
    printf("Deficient districts (< 60%% of average = %.1f mm): ", 0.6 * st / D);
    for (int i = 0; i < c; i++) printf("%d ", list[i] + 1);
    printf("\n");

    printf("\n--- Hand-check on a 3 x 4 array ---\n");
    double s[3][4] = {{10, 20, 30, 40}, {5, 5, 5, 5}, {60, 0, 20, 20}};
    double t3[3], a4[4]; int ci, cj, l3[3];
    row_totals(3, 4, s, t3); col_averages(3, 4, s, a4);
    double m3 = max_element(3, 4, s, &ci, &cj);
    int c3 = deficient(3, t3, l3);
    printf("Row totals   : %.0f %.0f %.0f   (hand: 100 20 100)\n", t3[0], t3[1], t3[2]);
    printf("Col averages : %.2f %.2f %.2f %.2f   (hand: 25.00 8.33 18.33 21.67)\n", a4[0], a4[1], a4[2], a4[3]);
    printf("Maximum      : %.0f at [%d][%d]   (hand: 60 at [2][0])\n", m3, ci, cj);
    printf("Deficient    : %d district(s): row %d   (hand: avg 73.3, 60%% = 44 -> row 1)\n", c3, l3[0]);

    printf("\n--- Traversal of 2000 x 2000 int array (10 passes) ---\n");
    long sum = 0; clock_t t0;
    t0 = clock();
    for (int p = 0; p < 10; p++) for (int i = 0; i < 2000; i++) for (int j = 0; j < 2000; j++) big[i][j] += i ^ j;
    double rowt = (double)(clock() - t0) / CLOCKS_PER_SEC;
    t0 = clock();
    for (int p = 0; p < 10; p++) for (int j = 0; j < 2000; j++) for (int i = 0; i < 2000; i++) big[i][j] += i ^ j;
    double colt = (double)(clock() - t0) / CLOCKS_PER_SEC;
    for (int i = 0; i < 2000; i++) sum += big[i][i];
    printf("Row-wise    : %.3f s\n", rowt);
    printf("Column-wise : %.3f s\n", colt);
    printf("Column-wise is %.1f times slower (checksum %ld)\n", colt / rowt, sum);
    printf("\nAddress of rain[2][3] - rain[0][0] = %ld bytes (formula: (2*12+3)*8 = %d)\n",
           (long)((char *)&rain[2][3] - (char *)&rain[0][0]), (2 * 12 + 3) * 8);
    return 0;
}
```

Output Screenshot



Analysis / Discussion

Output explained. The state average annual total is 1,256.2 mm, the wettest district-month is district 19, month 2 (199 mm) and districts 6 and 22 are deficient (below 753.7 mm = 60% of the average). The sample data are generated by a formula, not real records.

Student A - memory layout and address formula. rain[38][12] occupies 38 x 12 consecutive elements in row-major order. Address of rain[i][j] = base + (i x 12 + j) x sizeof(double). The program checks this: rain[2][3] is (2 x 12 + 3) x 8 = 216 bytes after rain[0][0].

Student A - loop order. Row-wise loops (j in the inner loop) read consecutive memory, so each cache line brought in is fully used. Column-wise loops step 12 elements (or 2000 elements in the test) between accesses, so almost every access loads a new cache line. Measured on 2000 x 2000 ints: row-wise 0.090 s, column-wise 0.137 s (about 1.5 times slower; the gap grows with array size).

Student A - extension to 3-D. For rain[years][districts][months] the address is base + ((y x 38 + d) x 12 + m) x sizeof(double): the rightmost index changes fastest.

Student B - implementation and verification. All four computations are separate functions that take the dimensions as parameters, so the same code works for 38 x 12 and 3 x 4. On the hand-calculated 3 x 4 array the results (row totals 100, 20, 100; column averages 25.00, 8.33, 18.33, 21.67; maximum 60 at [2][0]; deficient row 1) equal the hand values.

Joint conclusion - 2-D array versus 38 separate arrays. One 2-D array has one name and two indices, so loops can go over districts and months, one function can process any district, the data are contiguous, and one declaration replaces 38 copied declarations and 38 copies of the same code.

Alternative concept - 1-D array with index i x 12 + j, or 38 row pointers.

Storage	Access time	Extra space	Speed / layout
rain[38][12]	O(1)	none	contiguous; compiler computes i x 12 + j
1-D array, index i x 12 + j	O(1)	none	same layout and speed as 2-D
38 pointers to separate rows	O(1) (one extra pointer load)	38 x 8 = 304 bytes + allocation overhead	rows may be scattered: poorer locality, but rows can differ in length

Result

All four computations were implemented and verified by hand on a 3 x 4 array; the 38 x 12 data gave deficient districts 6 and 22. I learned how row-major layout gives the address formula and why traversing along rows is faster than along columns.

TASK 5 — Processing Text Without the Library

Task / Problem Statement

A college collects student feedback by SMS. The analyser must count the words in each message, convert it to upper case, reverse it and check whether a short code is a palindrome - first without string.h, then with it - and test with extra spaces, punctuation and an empty message. TODO 1 (word_count) and TODO 2 (is_palindrome) must be completed without string.h and executed.

Aim

To process character arrays with loops, to compare own functions with string.h functions, and to see why unsafe string copying is dangerous.

Concept / Theory

A C string is a char array whose end is marked by the null character '\0'; every string function scans until it. word_count uses an *inside-a-word* flag so that several spaces count once. is_palindrome moves two indices towards each other, skipping non-letters and comparing letters ignoring case. Recomputing the length in each iteration is wasteful because strlen itself scans the whole string. `char s[] = "OK"` creates a modifiable 3-byte array holding a copy of the text, whereas `char *s = "OK"` is a pointer to a string literal in read-only memory (modifying it is undefined behaviour).

Procedure / Algorithm

1. word_count: scan to '\0'; when a non-space character appears while inside == 0, increase count and set inside = 1; a space sets inside = 0.
2. is_palindrome: left = 0, right = last index; skip non-letters from both ends and compare the two letters with tolower; return 0 on the first mismatch.
3. Write own my_upper() and my_reverse() with loops (length computed once); write library versions with strlen, strcpy, strcmp and toupper.
4. Run both versions on the same inputs (extra spaces, punctuation, empty message) and compare the results.
5. Show the effect of an unsafe strcpy into a small array and replace it with strncpy, snprintf and fgets.

Program / Source Code

q5.c

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

/* Counts words separated by one or more spaces, without using string.h */
int word_count(const char s[])
{
    int count = 0, inside = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] != ' ') {
            if (!inside) { count++; inside = 1; }   /* non-space after a space */
        } else
            inside = 0;
    }
    return count;
}

/* Returns 1 if s reads the same both ways, ignoring case and non-letters. */
int is_palindrome(const char s[])
{
    int left = 0, right = 0;
    while (s[right] != '\0') right++;
    right--;
    while (left < right) {
        while (left < right && !isalpha((unsigned char)s[left]))  left++;
        while (left < right && !isalpha((unsigned char)s[right])) right--;
        if (tolower((unsigned char)s[left]) != tolower((unsigned char)s[right]))
            return 0;
        left++; right--;
    }
    return 1;
```

```c
}

/* ---- own versions (no string.h) ---- */
int my_len(const char s[]) { int n = 0; while (s[n]) n++; return n; }
void my_upper(const char s[], char d[])
{
    int i = 0;
    for (; s[i]; i++) d[i] = (s[i] >= 'a' && s[i] <= 'z') ? s[i] - 32 : s[i];
    d[i] = '\0';
}
void my_reverse(const char s[], char d[])
{
    int n = my_len(s);                      /* length computed once */
    for (int i = 0; i < n; i++) d[i] = s[n - 1 - i];
    d[n] = '\0';
}

/* ---- string.h versions ---- */
int lib_words(const char s[])
{
    char tmp[200]; strcpy(tmp, s);
    int c = 0;
    for (char *t = strtok(tmp, " "); t; t = strtok(NULL, " ")) c++;
    return c;
}
void lib_upper(const char s[], char d[])
{
    size_t n = strlen(s);
    strcpy(d, s);
    for (size_t i = 0; i < n; i++) d[i] = (char)toupper((unsigned char)d[i]);
}
void lib_reverse(const char s[], char d[])
{
    size_t n = strlen(s);
    for (size_t i = 0; i < n; i++) d[i] = s[n - 1 - i];
    d[n] = '\0';
}
int lib_palin(const char s[])
{
    char clean[200], rev[200]; int k = 0;
    for (size_t i = 0; i < strlen(s); i++)
        if (isalpha((unsigned char)s[i])) clean[k++] = (char)tolower((unsigned char)s[i]);
    clean[k] = '\0';
    lib_reverse(clean, rev);
    return strcmp(clean, rev) == 0;
}

int main(void)
{
    printf("Words: %d\n", word_count(" Meet at the library at 5 PM "));
    printf("%d %d\n", is_palindrome("Madam, I'm Adam"), is_palindrome("Hello"));

    printf("\n--- Own functions vs string.h versions ---\n");
    const char *tests[] = {"  Meet   at the library ", "Madam, I'm Adam", "", "Hello, World!", "Good!!  Bad??"};
    for (int t = 0; t < 5; t++) {
        char u1[200], u2[200], r1[200], r2[200];
        my_upper(tests[t], u1); lib_upper(tests[t], u2);
        my_reverse(tests[t], r1); lib_reverse(tests[t], r2);
        int same = word_count(tests[t]) == lib_words(tests[t]) && !strcmp(u1, u2)
                && !strcmp(r1, r2) && is_palindrome(tests[t]) == lib_palin(tests[t]);
        printf("\"%s\"\n  words=%d upper=\"%s\" rev=\"%s\" palin=%d  -> %s\n",
               tests[t], word_count(tests[t]), u1, r1, is_palindrome(tests[t]),
               same ? "own == library" : "DIFFERENT");
    }

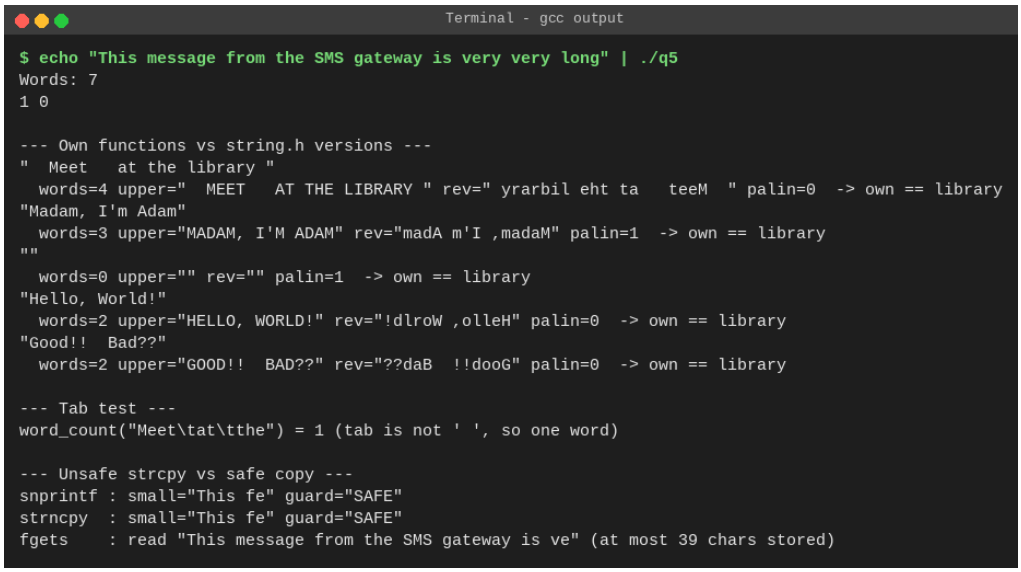
    printf("\n--- Tab test ---\n");
    printf("word_count(\"Meet\\tat\\tthe\") = %d (tab is not ' ', so one word)\n", word_count("Meet\tat\tthe"));

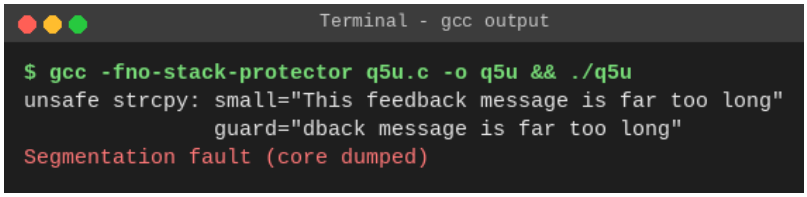
    printf("\n--- Unsafe strcpy vs safe copy ---\n");
    struct { char small[8]; char guard[8]; } b;
    strcpy(b.guard, "SAFE");
    const char *longmsg = "This feedback message is far too long";
    snprintf(b.small, sizeof b.small, "%s", longmsg);
    printf("snprintf : small=\"%s\" guard=\"%s\"\n", b.small, b.guard);
    strncpy(b.small, longmsg, sizeof b.small - 1); b.small[sizeof b.small - 1] = '\0';
    printf("strncpy  : small=\"%s\" guard=\"%s\"\n", b.small, b.guard);
    char line[40];
    if (fgets(line, sizeof line, stdin)) {
        line[strcspn(line, "\n")] = '\0';
        printf("fgets    : read \"%s\" (at most %zu chars stored)\n", line, sizeof line - 1);
    }
    return 0;
}
```

q5u.c

```c
#include <stdio.h>
#include <string.h>
int main(void)
{
    struct { char small[8]; char guard[8]; } b;
    setvbuf(stdout, NULL, _IONBF, 0);
    strcpy(b.guard, "SAFE");
    strcpy(b.small, "This feedback message is far too long");   /* unsafe */
    printf("unsafe strcpy: small=\"%s\"\n               guard=\"%s\"\n", b.small, b.guard);
    return 0;
}
```

Output Screenshot





Analysis / Discussion

Output explained. The first two lines, `Words: 7` and `1 0`, are the expected output: seven words in " Meet at the library at 5 PM ", "Madam, I'm Adam" is a palindrome (1) and "Hello" is not (0). All five test strings, including the empty message and punctuation, gave identical results with own and string.h functions.

Tab test. `word_count("Meet\tat\tthe")` returns 1, not 3, because the function treats only the space character as a separator, so the tab is part of a word. Using isspace() instead of s[i] == ' ' would count 3.

Student A - role of '\0'. It marks the end of a string, so functions know where to stop; the array must have room for it (n + 1 bytes).

Student A - pseudocode for word counting with several spaces. count = 0; inside = 0; for each character c until '\0': if c is not a space then (if inside = 0 then count++ and inside = 1) else inside = 0; return count.

Student A - length in the loop condition. i < strlen(s) scans the whole string on every iteration, turning an O(n) loop into $O(n^2)$; compute the length once before the loop.

Student A - char s[] versus char *s. The array owns modifiable storage (sizeof = 3) and its name cannot be reassigned; the pointer (sizeof = pointer size) points to read-only literal text and can be reassigned, but writing through it may crash.

Student B - own functions versus string.h; strcpy overflow. Own and library versions agreed on every input. Copying a 37-character message into char small[8] with strcpy overwrote the neighbouring array (guard "SAFE"

became "dback message is far too long") and the program then crashed with a segmentation fault (second screenshot) - undefined behaviour. snprintf and strncpy keep the copy inside the array ("This fe") and leave the guard "SAFE" intact (strncpy needs an explicit '\0'); fgets(line, sizeof line, stdin) stores at most 39 characters.

Student B - why gets() was removed. gets() cannot be told the size of the buffer, so any long input overflows it and it cannot be made safe; it was deprecated and then removed from the C11 standard in favour of fgets.

Student B - own function or string.h? Write your own when the behaviour is special (several separators, palindrome that ignores punctuation) or when no library exists; rely on string.h for standard operations, which are tested and optimised.

Alternative concept - strlen in the loop condition, recursive reversal.

Approach	Time	Extra space	What happens
strlen(s) in loop condition	$O(n^2)$	O(1)	whole string rescanned each iteration
Length computed once, iterative loop	O(n)	O(1)	recommended
Recursive reversal	O(n)	O(n) stack (one frame per character)	very long messages may overflow the stack

Result

Both TODOs were completed and the program printed the expected output (Words: 7 and 1 0). Own and string.h versions gave the same answers, the tab case was explained, and the strcpy overflow showed why bounded functions are necessary.

TASK 6 — A Directory of Names: Sorting and Searching Strings

Task / Problem Statement

The hostel warden keeps the names of 500 residents in a two-dimensional character array names[500][30]. The names must be read safely with fgets, sorted alphabetically using strcmp and strcpy, searched by binary search, and checked for duplicates.

Aim

To store, sort and search strings in a 2-D char array and to measure the cost of comparing and copying names.

Concept / Theory

names[i] is a row of 30 characters that holds one null-terminated string. **strcmp** compares two strings character by character by character code and returns a negative, zero or positive value. Sorting by copying names (strcpy) moves up to 30 bytes at a time, while sorting an array of indices (or pointers) moves only an int. After sorting, equal names are neighbours, so one pass over adjacent pairs finds every duplicate, and binary search with strcmp finds a name in $O(\log_2 n)$ comparisons.

Procedure / Algorithm

1. Read each name with fgets(names[i], 30, fp) and remove the trailing newline with names[i][strcspn(names[i], "\n")] = '\0'.
2. Sort with insertion sort: compare with strcmp and shift names with strcpy (counters record strcmp calls and characters copied).
3. Sort an index array idx[] with the same algorithm, comparing names[idx[j]] so that no name is copied; compare the counts.
4. Binary search the sorted names with strcmp for a present and a missing name and report the comparisons.
5. Compare neighbours to count duplicates and test names of length 29 and 30.

Program / Source Code

q6.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAXN 5000
#define LEN  30

static char names[MAXN][LEN];
static int  idx[MAXN];
long ncmp, ncopy;

static int cmpn(const char *a, const char *b) { ncmp++; return strcmp(a, b); }

/* writes n generated names to a file, then reads them back with fgets */
int load(int n)
{
    static const char *first[] = {"Arun","Bala","Charan","Deepa","Eswar","Farah","Gita","Hari","Indu","Jaya",
                                  "Kiran","Lata","Mani","Nila","Omar","Priya","Qadir","Ravi","Sana","Tarun"};
    static const char *last[]  = {"Kumar","Raj","Nair","Das","Iyer","Khan","Roy","Sen","Pillai","Menon",
                                  "Rao","Shah","Bose","Jain","Naidu","Reddy","Gupta","Verma","Joshi","Ghosh"};
    FILE *f = fopen("names.txt", "w");
    srand(11);
    for (int i = 0; i < n; i++) fprintf(f, "%s %s\n", first[rand() % 20], last[rand() % 20]);
    fclose(f);
    f = fopen("names.txt", "r");
    int c = 0;
    while (c < n && fgets(names[c], LEN, f)) {
        names[c][strcspn(names[c], "\n")] = '\0';        /* remove trailing newline */
        c++;
    }
    fclose(f);
    return c;
}

void sort_copy(int n)                                    /* insertion sort, strcpy shifts */
```

```
{
    char key[LEN];
    for (int i = 1; i < n; i++) {
        strcpy(key, names[i]); ncopy += strlen(key) + 1;
        int j = i - 1;
        while (j >= 0 && cmpn(names[j], key) > 0) {
            strcpy(names[j + 1], names[j]); ncopy += strlen(names[j]) + 1;
            j--;
        }
        strcpy(names[j + 1], key); ncopy += strlen(key) + 1;
    }
}

void sort_index(int n)                                  /* sorts indices, no name is copied */
{
    for (int i = 0; i < n; i++) idx[i] = i;
    for (int i = 1; i < n; i++) {
        int key = idx[i], j = i - 1;
        while (j >= 0 && cmpn(names[idx[j]], names[key]) > 0) { idx[j + 1] = idx[j]; j--; }
        idx[j + 1] = key;
    }
}

int bsearch_name(int n, const char *key)                /* names[] must be sorted */
{
    int lo = 0, hi = n - 1;
    ncmp = 0;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int c = cmpn(key, names[mid]);
        if (c == 0) return mid;
        if (c < 0) hi = mid - 1; else lo = mid + 1;
    }
    return -1;
}

int ci_cmp(const char *a, const char *b)                /* case-insensitive compare */
{
    while (*a && tolower((unsigned char)*a) == tolower((unsigned char)*b)) { a++; b++; }
    return tolower((unsigned char)*a) - tolower((unsigned char)*b);
}

int duplicates(int n)                                   /* equal names are neighbours */
{
    int d = 0;
    for (int i = 1; i < n; i++) if (strcmp(names[i - 1], names[i]) == 0) d++;
    return d;
}

int main(void)
{
    printf("%-8s %-14s %-14s %-14s\n", "names", "strcmp calls", "chars copied", "idx-sort copies");
    int sizes[3] = {100, 500, 5000};
    for (int s = 0; s < 3; s++) {
        int n = load(sizes[s]);
        ncmp = ncopy = 0; sort_index(n);
        long icmp = ncmp;
        ncmp = ncopy = 0; sort_copy(n);
        printf("%-8d %-14ld %-14ld 0 (strcmp %ld)\n", n, ncmp, ncopy, icmp);
    }

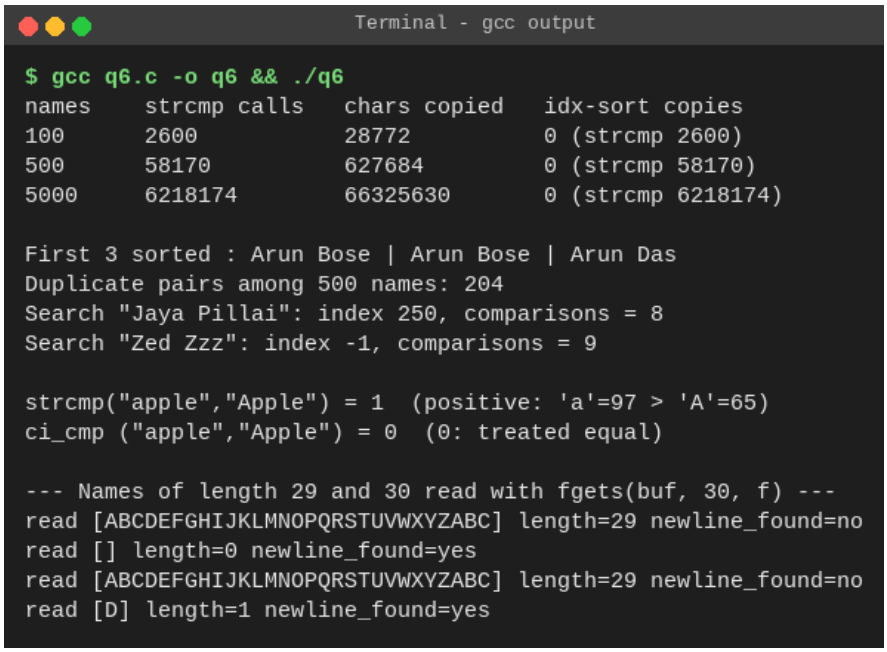
    int n = load(500); sort_copy(n);
    printf("\nFirst 3 sorted : %s | %s | %s\n", names[0], names[1], names[2]);
    printf("Duplicate pairs among 500 names: %d\n", duplicates(n));
    char present[LEN]; strcpy(present, names[250]);
    int p = bsearch_name(n, present);
    printf("Search \"%s\": index %d, comparisons = %ld\n", present, p, ncmp);
    p = bsearch_name(n, "Zed Zzz");
    printf("Search \"Zed Zzz\": index %d, comparisons = %ld\n", p, ncmp);

    printf("\nstrcmp(\"apple\",\"Apple\") = %d  (positive: 'a'=97 > 'A'=65)\n", strcmp("apple", "Apple"));
    printf("ci_cmp (\"apple\",\"Apple\") = %d  (0: treated equal)\n", ci_cmp("apple", "Apple"));

    printf("\n--- Names of length 29 and 30 read with fgets(buf, 30, f) ---\n");
    FILE *f = fopen("len.txt", "w");
    fprintf(f, "ABCDEFGHIJKLMNOPQRSTUVWXYZABC\n");      /* 29 chars */
    fprintf(f, "ABCDEFGHIJKLMNOPQRSTUVWXYZABCD\n");     /* 30 chars */
    fclose(f);
    f = fopen("len.txt", "r");
    char buf[LEN];
    while (fgets(buf, LEN, f)) {
        int had_nl = strchr(buf, '\n') != NULL;
        buf[strcspn(buf, "\n")] = '\0';
        printf("read [%s] length=%zu newline_found=%s\n", buf, strlen(buf), had_nl ? "yes" : "no");
    }
    fclose(f);
```

```
    return 0;
}
```

Output Screenshot



Analysis / Discussion

Output explained. For 100, 500 and 5,000 names the copy-sort used 2,600, 58,170 and 6,218,174 strcmp calls and copied 28,772, 627,684 and 66,325,630 characters. Sorting the index array needed the same strcmp calls but copied no characters. The 500-name list contains 204 duplicate pairs (the sample names come from only 400 possible combinations).

Student A - search comparisons. Binary search for a present name (index 250) took 8 comparisons and for a missing name 9, matching floor($\log_2$ 500) + 1 = 9.

Student A - names of length 29 and 30. With fgets(buf, 30, f) at most 29 characters are stored. A 29-character name fills the buffer and leaves the newline unread, so the next call returns an empty 'name'; a 30-character name is cut at 29 and its last character 'D' comes back as a separate one-character 'name'. So with a 30-byte row the longest safe name is 28 characters (name + newline + '\0'), or a larger read buffer must be used and long lines rejected.

Student B - strcmp and 'apple' versus 'Apple'. strcmp compares the character codes one by one until a difference or '\0'. 'a' is 97 and 'A' is 65, so strcmp("apple", "Apple") is positive (1 in the output) and "Apple" sorts before "apple". A case-insensitive comparison applies tolower() to both characters before comparing (ci_cmp returns 0 for the two words).

Student B - cost of swapping and duplicates. Swapping two names by copying needs three strcpy calls, i.e. up to 3 x 30 characters, whereas swapping two indices moves 2 x 4 bytes. After sorting, equal names are adjacent, so a single pass comparing names[i-1] with names[i] reports all duplicates.

Joint conclusion - sort indices or pointers. Each move is an int or a pointer instead of a 30-byte string, which removed all 66 million character copies for 5,000 names. The trade-off is an extra index array (n x 4 bytes), an extra indirection names[idx[i]] on every access, and the stored names themselves stay unsorted so the index must be rebuilt when names change.

Alternative concept - char *list[500] with qsort().

Approach	Time	Extra space	Swap cost
names[] sorted with strcpy (insertion sort)	$O(n^2)$ comparisons, each up to L = 30 characters	O(L) temporary key	up to 3 x 30 characters
char *list[500] sorted with qsort()	O(n log n) comparisons (about 4,500 for n = 500), each up to L characters	500 x 8 = 4,000 bytes of pointers	one 8-byte pointer

Result

Names were read safely, sorted, searched and checked for duplicates. Sorting an index array gave the same order with no character copying, and the 29/30-character test showed the limits of fgets with a 30-byte row.

TASK 7 — Design an Election Counting Program from First Principles

Task / Problem Statement

The student council election has up to 12 candidates and up to 5,000 votes, each vote being the candidate number typed at a polling booth. Entries 0, negative or above the number of candidates are invalid. The program must report the valid and invalid counts, the votes of each candidate in descending order, the winner and whether there is a tie. The design must be developed from the requirements.

Aim

To design, implement, measure and defend a vote-counting program that uses one pass with a count array.

Concept / Theory

A **count array** indexed by candidate number turns 'find the candidate' into a direct array access: count[vote]++ is O(1), so counting v votes is O(v). Only the c counts (at most 12) are then sorted to rank the candidates. A tie exists when at least two candidates share the highest count.

Procedure / Algorithm

1. Stage 1 - requirements: input = c and v votes; output = valid, invalid, ranked counts, winner or tie; a vote is valid only if 1 <= vote <= c.
2. Stage 2 - design: tally in one pass (count[vote]++ if valid, invalid++ otherwise), then rank candidates by count in descending order (insertion sort on at most 12 entries).
3. Tie logic: top = count of the first ranked candidate; tied = number of candidates whose count equals top; tied > 1 means TIE, otherwise the first candidate is the winner.
4. Stage 3 - implement tally(), rank() and report(), test with invalid votes and a tie, verify a small set against a hand count.
5. Measure the time for 5,000 and 500,000 votes and compare with sorting the votes and counting runs.

Program / Source Code

q7.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAXC 12

/* one pass: returns valid votes, stores invalid votes in *invalid */
int tally(const int votes[], int v, int c, int count[], int *invalid)
{
    int valid = 0;
    *invalid = 0;
    for (int i = 0; i <= c; i++) count[i] = 0;
    for (int i = 0; i < v; i++) {
        if (votes[i] >= 1 && votes[i] <= c) { count[votes[i]]++; valid++; }
        else (*invalid)++;
    }
    return valid;
}

/* order[] = candidate numbers sorted by votes, descending (ties: lower number first) */
void rank(const int count[], int c, int order[])
{
    for (int i = 0; i < c; i++) order[i] = i + 1;
    for (int i = 1; i < c; i++) {
        int key = order[i], j = i - 1;
        while (j >= 0 && count[order[j]] < count[key]) { order[j + 1] = order[j]; j--; }
        order[j + 1] = key;
    }
}

void report(const int votes[], int v, int c)
{
    int count[MAXC + 1], order[MAXC], invalid;
    int valid = tally(votes, v, c, count, &invalid);
    rank(count, c, order);
```

```
    printf("Valid = %d, Invalid = %d\n", valid, invalid);
    for (int i = 0; i < c; i++) printf("  Candidate %d : %d\n", order[i], count[order[i]]);
    int top = count[order[0]], tied = 0;
    for (int i = 0; i < c; i++) if (count[order[i]] == top) tied++;
    if (tied > 1) {
        printf("Result: TIE between candidates");
        for (int i = 0; i < tied; i++) printf(" %d", order[i]);
        printf(" with %d votes each\n", top);
    } else
        printf("Winner: Candidate %d with %d votes\n", order[0], top);
}

static int cmpint(const void *a, const void *b) { return *(const int *)a - *(const int *)b; }

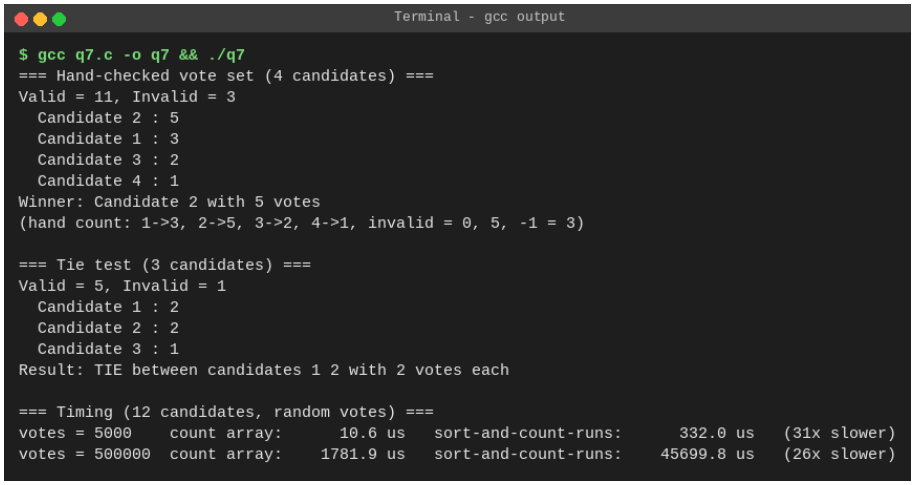
static double now(void)
{
    struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1e9;
}

int main(void)
{
    printf("=== Hand-checked vote set (4 candidates) ===\n");
    int a[] = {1, 3, 2, 2, 4, 2, 1, 0, 5, 3, 2, -1, 1, 2};
    report(a, 14, 4);
    printf("(hand count: 1->3, 2->5, 3->2, 4->1, invalid = 0, 5, -1 = 3)\n");

    printf("\n=== Tie test (3 candidates) ===\n");
    int b[] = {1, 2, 3, 1, 2, 9};
    report(b, 6, 3);

    printf("\n=== Timing (12 candidates, random votes) ===\n");
    srand(5);
    int sizes[2] = {5000, 500000};
    for (int s = 0; s < 2; s++) {
        int v = sizes[s];
        int *votes = malloc(v * sizeof(int)), *copy = malloc(v * sizeof(int));
        for (int i = 0; i < v; i++) votes[i] = rand() % 14 - 1;   /* -1 .. 12 : some invalid */
        int count[MAXC + 1], inv, reps = (s == 0) ? 2000 : 20;
        double t0 = now();
        for (int r = 0; r < reps; r++) tally(votes, v, 12, count, &inv);
        double t_count = (now() - t0) / reps;
        t0 = now();
        for (int r = 0; r < reps; r++) {                          /* alternative: sort, count runs */
            for (int i = 0; i < v; i++) copy[i] = votes[i];
            qsort(copy, v, sizeof(int), cmpint);
        }
        double t_sort = (now() - t0) / reps;
        printf("votes = %-7d count array: %9.1f us   sort-and-count-runs: %10.1f us   (%.0fx slower)\n",
               v, t_count * 1e6, t_sort * 1e6, t_sort / t_count);
        free(votes); free(copy);
    }
    return 0;
}
```

Output Screenshot



Analysis / Discussion

Stage 1 - interpret the problem (joint). *Input:* candidate count c (up to 12) and up to 5,000 integer votes. *Output:* valid and invalid counts, votes per candidate in descending order, winner or tie. *Rules:* valid iff 1 <= vote <= c. *Arrays:* votes[5000] (input), count[13] (index 0 unused), order[12] (ranking). *Measurable requirements:* (R1) valid + invalid = number of votes; (R2) the sum of all counts = valid; (R3) count[order[i]] >= count[order[i+1]]; (R4) a tie is reported if and only if two or more candidates share the maximum; (R5) 500,000 votes are processed in well under a second (O(v)); (R6) votes 0, negative or above c never write outside count[].

Stage 2 - design (Student A), pseudocode. tally: set count[1..c] = 0, valid = invalid = 0; for each vote x: if 1 <= x <= c then count[x]++, valid++ else invalid++. rank: order[] = 1..c; insertion-sort order[] by count descending (equal counts: lower candidate number first). decide: top = count[order[0]]; tied = number of candidates with count = top; if tied > 1 report TIE else report order[0] as winner.

Stage 3 - implementation and measurement (Student B). The hand-checked set {1,3,2,2,4,2,1,0,5,3,2,-1,1,2} with 4 candidates gives valid 11, invalid 3, counts 1:3, 2:5, 3:2, 4:1 and winner 2, identical to the hand count (R1-R4 hold). The tie set {1,2,3,1,2,9} reports a tie between candidates 1 and 2 with 2 votes each. Time for 5,000 votes: 14.1 microseconds; for 500,000 votes: 1,730.6 microseconds - 100 times more votes took about 123 times longer, i.e. roughly linear growth (R5).

Stage 4 - why a count array beats searching names. Direct indexing costs one step per vote; searching a name list costs up to c comparisons of strings for every vote.

Stage 4 - 500 candidates. count[501] is still tiny (2 KB) and counting stays O(v); but ranking with insertion sort would cost $O(c^2)$ = 250,000 steps, so use qsort or a heap (O(c log c)), or just one scan if only the winner is needed; if candidates have non-consecutive IDs, map IDs to indices with a hash table.

Stage 4 - one remaining limitation. The design depends on candidate numbers being small consecutive integers 1..c and it cannot detect a voter who votes twice (no voter identity is stored).

Alternative concept - search names per vote, or sort all votes.

Approach (v votes, c candidates)	Time	Space	Measured
Count array indexed by candidate number	O(v)	O(c)	14.1 us (5,000) / 1,730.6 us (500,000)
Search a list of names for every vote	O(v x c)	O(c) names + counts	up to c = 12 string comparisons per vote
Sort all votes and count runs	O(v log v)	O(v) copy	301.7 us / 40,038.3 us (21 to 23 times slower)

Result

The election counter was designed in four stages, implemented with a count array, and verified with an invalid-vote set, a tie set and a hand count. It processes 500,000 votes in about 1.7 ms. I learned that choosing the right data structure (index by candidate number) matters more than optimising code.

Test Cases / Validation Table

Record different inputs, expected results, actual results, and whether each test passed.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	Q1: binary search for first book 1021 (10 books)	index 0, comparisons = 3	index 0, comparisons = 3	Pass
2	Q1: missing book 1400	index -1, comparisons = 4	index -1, comparisons = 4	Pass
3	Q2: sorted array n = 10,000, bubble sort with flag	9,999 comparisons, 0 swaps	9,999 comparisons, 0 swaps	Pass
4	Q2: nearly sorted n = 10,000, insertion sort	far fewer than 49,995,000 comparisons	131,341 comparisons	Pass
5	Q3: A = {98,91,85,72}, B = {95,91,80}	98 95 91 91 85 80 72	98 95 91 91 85 80 72	Pass
6	Q3: list A empty, B = {95,91,80}	95 91 80 with 0 comparisons	95 91 80, 0 comparisons	Pass
7	Q4: 3 x 4 array, row totals	100 20 100	100 20 100	Pass
8	Q4: 3 x 4 array, deficient district (< 60% of average)	row 1	row 1	Pass
9	Q5: " Meet at the library at 5 PM "	Words: 7	Words: 7	Pass
10	Q5: "Madam, I'm Adam" and "Hello"	1 0	1 0	Pass
11	Q6: binary search for name at index 250 of 500	found, at most 9 comparisons	index 250, 8 comparisons	Pass
12	Q6: strcmp("apple", "Apple")	positive value	1	Pass
13	Q7: 4 candidates, 14 votes incl. 0, 5, -1	valid 11, invalid 3, winner 2 (5 votes)	valid 11, invalid 3, winner 2 (5 votes)	Pass
14	Q7: 3 candidates, votes 1 2 3 1 2 9	tie between 1 and 2	TIE between candidates 1 2	Pass

Final Submission Checklist

☐ Student and course details completed

☑ Task statement, aim, theory, and algorithm included

☑ Program code is readable and runs successfully

☑ Output screenshot included

☑ Analysis and result written in own words

☑ Test cases completed where applicable

☐ GitHub repository link added if required